

Programación Orientada a Objetos 2

Trabajo Práctico Integrador

UNQ-Shop

Integrantes:

- Lezcano, Nahuel: nahuel20nhl@gmail.com
- Bogarín, Leandro: bogarinleandro@hotmail.com

Decisiones de diseño

1. Catálogo de productos.

El patrón utilizado es el **Composite**, porque trata de manera uniforme a objetos individuales y a conjuntos de objetos, definiendo una interfaz común, permitiendo que podamos tener productos por su cuenta, productos dentro de paquetes y paquetes dentro de otros paquetes.

Componente: Item - **Hoja:** Producto - **Compuesto:** Paquete.

2. Ciclo de vida del pedido.

Un pedido modifica su comportamiento a lo largo de su ciclo de vida. Por esta razón el patrón elegido fue el **State**. Es un patrón que permite a un objeto alterar su comportamiento cuando su estado interno cambia.

En este caso, el pedido posee estados: Borrador, Pago, En Preparación, Enviado, Entregado y Cancelado. Cada estado será el responsable de manejar la lógica interna.

Esto evita una clase contaminada con enormes condicionales que alteran el modo en que se comporta la clase.

Contexto: Pedido - **Estado:** Estado - **Estados Concretos:** Borrador, Pago, En Preparación, Enviado, Entregado y Cancelado.

3. Métodos de envío.

Se necesitaban estrategias diferentes para realizar el envío de un pedido, así que usamos **Strategy** porque nos permite definir diferentes estrategias dependiendo de lo que se quiera realizar.

Contexto: Envío - **Estrategia:** MetodoDeEnvío - **Estrategias Concretas:** EnvíoEstandar, EnvíoExpress y RetiroSucursal.

4. Métodos de pago.

El procesamiento de un pago siempre sigue los mismos pasos en el mismo orden. El patrón **Template Method** resuelve este problema. Se define un esqueleto de un algoritmo en la superclase, pero que permite a las subclases sobrescribir pasos del algoritmo sin cambiar su estructura.

Existe una clase abstracta **MetodoDePago** donde se implementa el procesamiento del pago, y a la vez agrupa los métodos necesarios. Cada medio de pago extiende esa clase, y sobrescribe los pasos para configurar a su modo las acciones.

Cada clase medio de pago le da la responsabilidad de sus métodos a unas interfaces específicas que sirven como APIs.

Clase Abstracta: MetodoDePago - **Clases Concretas:** TarjetaDeCrédito, TransferenciaBancaria, BilleteraVirtual.

5. Notificaciones del Pedido.

Usamos el patrón **Observer** porque nos permite notificar estados del pedido a objetos, haciendo que estos se actualicen automáticamente cuando el estado del pedido cambie.

Sujeto: Pedido - **Observador:** Observador - **Sujetos Concretos:** NotificadorDeMail, GeneradorDeFactura y Fidelizacion.

6. Búsqueda en el catálogo.

Se implementó el patrón **Composite** para la búsqueda en el catálogo, ya que podemos anidar las condiciones de búsqueda tratándolas como hojas y compuestos.

Componente: Criterio - **Hojas:** CriterioNombre, CriterioPrecioMax, CriterioCategoria y CriterioDisponibilidad - **Compuestos:** Conjuncion, Disyuncion y Negacion.

7. Reportes.

Para los reportes utilizamos el patrón **Visitor**, el reporte usa la tienda para generar archivos TXT, CSV y HTML, pero, como todo patrón, está hecho de forma que puedan agregarse más archivos siempre que implementen la clase abstracta Reporte.

Clase Abstracta: Reporte - **Clases Concretas:** ReporteTxt, ReporteCSV y ReporteHTML.